CampusConnect

Campus Gig & SkillSwap Portal — Complete Product Requirements Document

Hackathon duration: 5 hours 30 minutes | Team: 3 | Product: Responsive full-stack web MVP | Development: AI-assisted

Version 1.0 | Authoritative scope: all five supplied hackathon milestones

1. Product Overview

Problem. Students with programming, design, writing, editing and tutoring skills lack a centralized campus marketplace for short-term services, paid help, voluntary support and skill exchange.

Solution. A student marketplace for flexible gig posting and applying, discovery, applicant selection, controlled status transitions, mutual reviews and computed reputation. All data is fetched and mutated through a functioning seeded mock API.

Audience. University students who offer skills, request help or do both.

Objective	Success measure
Flexible participation	One student account can post and apply to different gigs
Gig marketplace	Validated Offering and Requesting gigs with three payment types
Reliable workflow	Apply, select one, complete and mutually review
Integration	All pages backed by persistent mock API and valid seed data

2. Scope and Priorities

Priority	Features
P0 — Mandatory	Profiles; gig creation; search and filters; applications; owner controls; automatic acceptance/status rules; mutual reviews; calculated reputation; mock API and seed data
P1 — Important	Responsive UI; dashboard; loading, empty and error states
P2 — Optional	Animations, dark mode

Out of scope: real payment processing, chat, AI matching, notifications, uploads, real authentication, university email verification. Do not spend hackathon time on these.

3. Roles and Permissions

All users have one flexible Student role. The gig creator is its Owner/Poster; another student may be an Applicant. A student can be an owner on one gig and an applicant on another.

Action	Owner	Other student / accepted applicant
View gig	Yes	Yes
Apply	No	Other student: only if eligible and Open
View all gig applications	Yes	No
Accept/reject applicants	Yes	No
Complete In Progress gig	Yes	No

Action	Owner	Other student / accepted applicant
Review other participant after completion	Yes	Accepted applicant: yes

Demo identity: Provide a switcher among 12 seeded users. Send selected identity in X-Demo-User-Id; the backend enforces owner/participant checks. This is demo-only identity simulation, not production security.

4. Functional Requirements

FR-01 Profiles

• Display name, major, semester, bio, skills, gigs posted, gigs completed and average received rating.

• Derive posted count from gigs.user_id. Derive completed count from completed gigs where student is owner or accepted applicant, counting each gig once.

• Average = sum of reviews.rating for reviewee / number of received reviews; round to two decimals. Display “No ratings yet” if none. Never permit manual rating edits.

FR-02 Create Gigs

Field	Validation
title	Required, 5–100 characters
description	Required, 20–2000 characters
type	Offering or Requesting
category	Programming, Design, Writing, Tutoring
payment_type	Paid, SkillSwap, Free
budget	Positive number for Paid; null otherwise
deadline	Future date/time

Server assigns id, user_id, created_at, status=Open and accepted_applicant_id=null. New gig appears in Explore and owner’s My Gigs.

FR-03 Discovery

Browse and combine category, type, payment type, minimum/maximum budget and case-insensitive keyword search over title and description. Budget range applies to Paid gigs. Show non-Open gigs but disable application actions. Open gigs past deadline appear as Expired and cannot receive applications.

FR-04 Applications

Eligible students submit a 10–500 character message. Server assigns applicant_id from demo identity, id, timestamp and Pending status. Reject missing gig, own gig, duplicate (even if previously rejected), non-Open gig, expired deadline and invalid message. My Applications displays each submitted gig and status.

FR-05 Owner Applicant Management

Owner can view applicant names, skills, messages and statuses. Owner can reject a Pending application. On acceptance, server validates owner, gig Open/not expired, and application Pending/belongs to gig; it then accepts selected application, rejects all other Pending applications, stores selected Applications.id in gig.accepted_applicant_id, changes gig to In Progress and blocks new applications. Persist this as one indivisible operation; never partially update.

FR-06 Status Lifecycle

From	Trigger	To
Open	Owner accepts one Pending applicant	In Progress
Open	Deadline passes	Expired
Open	Owner cancels	Cancelled
In Progress	Owner marks complete (accepted applicant required)	Completed

Only owners may change gig status. No direct Open-to-Completed transition; do not allow manually setting In Progress without selection. Expired, Cancelled and Completed are terminal for MVP. Evaluate expiry on the server and persist it when detected.

FR-07 Mutual Reviews

After completion, owner and accepted applicant can each rate the other 1–5 (integer) with a required comment. One review per reviewer per gig. No self-reviews, outsiders, pre-completion reviews or editing after submission. Reviewer ID is derived from demo identity.

5. Frontend Requirements

Design: Clean modern student marketplace; blue #2563EB, slate #0F172A, off-white #F8FAFC, green #10B981, amber #F59E0B. Responsive, readable and accessible. Every displayed action must work.

Page	Required content and actions
Dashboard	Welcome, profile summary, posted/completed counts, active applications, recent gigs
Explore Gigs	Search, filters, cards, status/payment badges, deadline, details link
Gig Details	Full information; eligible Apply form; owner applicant controls; completion or review controls when appropriate
Create Gig	Validated fields; conditional budget input; success/error feedback
My Gigs	Owned gigs, counts, applicants, acceptance/rejection, status/completion
My Applications	Submitted messages and Pending/Accepted/Rejected statuses
Student Profile	Academic details, skills, posted/completed counts, received reviews and calculated rating

Frontend uses React Router, reusable components and a centralized API client. Include loading, empty, validation and network-error states. Do not hardcode marketplace records in components.

6. Data Model

```
Users: id, name, major, semester, bio, skills[]
Gigs: id, user_id, title, description, type, category,
      payment_type, budget|null, deadline, status,
      accepted_applicant_id|null, created_at
Applications: id, gig_id, applicant_id, message, status, created_at
Reviews: id, gig_id, reviewer_id, reviewee_id, rating, comment
```

Important: gigs.accepted_applicant_id references Applications.id, NOT Users.id. Every foreign key must reference an existing record. One application per applicant per gig; at most one Accepted application per gig; one review per reviewer per gig. Reviews only for authorized participants of completed gigs.

Illustrative records

```
User: {"id":1,"name":"Sara Ahmed","major":"CS","semester":3,
       "bio":"Frontend developer","skills":["React","UI Design"]}
```

```
Gig: {"id":101,"user_id":1,"title":"Need React portfolio",
      "description":"Help build my student portfolio website",
      "type":"Requesting","category":"Programming",
      "payment_type":"Paid","budget":2000,
      "deadline":"2026-11-15T23:59:00.000Z","status":"Open",
      "accepted_applicant_id":null,"created_at":"2026-10-21T09:00:00Z"}
Application: {"id":201,"gig_id":101,"applicant_id":2,
              "message":"I can help with React development",
              "status":"Pending","created_at":"2026-10-21T10:00:00Z"}
Review: {"id":301,"gig_id":105,"reviewer_id":1,
         "reviewee_id":2,"rating":5,"comment":"Great work"}
```

Seed deadlines must be generated relative to the actual demo date, not copied blindly from examples.

7. Architecture and API Contract

Stack: React + Vite + Tailwind CSS + React Router; Node.js + Express; persistent JSON mock database; Fetch API; GitHub. Browser → HTTP API → Express validation/workflow services → JSON store.

```
campusconnect/
  frontend/src/{components,pages,services/api.js,App.jsx}
  backend/{server.js,routes/,middleware/demoAuth.js,
           services/,data/db.json,seed.js,package.json}
  README.md
```

Use consistent response envelope: {"success":true,"data":{},"message":"..."}. Errors: {"success":false,"error":{"code":"...","message":"..."}}.

Method	Endpoint	Purpose
GET	/users	Demo users
GET	/users/:id	Profile and computed stats
PATCH	/users/:id	Edit own allowed profile fields
GET	/gigs	Search and combined filters
GET	/gigs/:id	Gig detail
POST	/gigs	Create gig
PATCH	/gigs/:id	Allowed owner edit/status action
GET	/applications?gig_id=101	Owner views gig applications
GET	/applications?mine=true	Current user’s applications
POST	/applications	Submit application
PATCH	/applications/:id	Owner accepts/rejects
GET	/reviews?reviewee_id=2	Received reviews
GET	/reviews?gig_id=105	Gig reviews
POST	/reviews	Submit review

Example requests

```
GET /gigs?category=Programming&type=Requesting&payment_type=Paid
    &min_budget=500&max_budget=3000&q=React
POST /applications  {"gig_id":101,"message":"I can help..."}
PATCH /applications/201  {"action":"accept"}
PATCH /gigs/101  {"action":"complete"}
POST /reviews  {"gig_id":105,"reviewee_id":2,"rating":5,
                "comment":"Excellent collaboration"}
```

Authenticated demo actions use X-Demo-User-Id. Never trust client-supplied owner/reviewer/applicant identity. Use HTTP 200/201 for success; 400 invalid input, 401 missing demo identity, 403 forbidden, 404 missing resource, 409 duplicate/conflict, 500 unexpected failure.

Persist successful mutations to db.json. For single-process demo, serialize read-modify-write operations and atomically replace the file after writing a temporary copy, avoiding partial updates/races.

8. Seed Data

Collection	Minimum	Suggested
Users	12	12
Gigs	25	25
Applications	20	25
Reviews	8	10

Suggested categories: Programming 8, Design 6, Writing 6, Tutoring 5. Suggested statuses: Open 12, In Progress 5, Completed 5, Expired 2, Cancelled 1. Include both Offering/Requesting and all payment types. Ensure all 8+ reviews reference completed gigs with accepted applicants. Repeatable seed script resets to a known valid state.

9. Nonfunctional Requirements

• Responsive desktop/tablet/mobile UI with accessible labels and keyboard-friendly navigation.

• Clear loading, empty, success, validation and network-error states.

• All data mutations through mock API; refresh must preserve changes.

• Backend enforces authorization, validation, uniqueness and status transitions.

• No secrets or real credentials in GitHub; documented local setup.

• Use exact consistent field/enum names across frontend and backend.

10. Team Responsibilities

Member	Primary responsibilities
Frontend developer	React pages, components, responsive styling, API integration support
Backend developer	Express, JSON store, seed script, validation, lifecycle and review rules
Team lead	Repository, API contract, integration, end-to-end testing, bug triage, README, demo/submission

Agree before coding on field names, enums, response format, demo identity header, directories, branch ownership and integration bug ownership. Suggested branches: frontend, backend, integration; team lead merges tested work into main.

11. 330-Minute Execution Plan

Time	Activity / checkpoint
00:00–00:20	Repo setup, roles, API contract and directory structure
00:20–01:35	Parallel frontend, backend and seed implementation
01:35–02:55	Gig creation, filters, applications and selection workflow

Time	Activity / checkpoint
02:55–04:00	Integration, profiles, mutual reviews and calculated ratings
04:00–04:50	End-to-end tests and fixes; stop adding decorative features
04:50–05:30	README, submission, demo rehearsal, deployment if required

Checkpoints: by 90 minutes frontend/backend run and gigs fetch; by 180 minutes post→apply→accept works; by 240 minutes full lifecycle/reviews work; by 290 minutes all P0 tests pass.

12. Acceptance Test Matrix

ID	Scenario	Expected
01	Create valid gig	Open gig saved
02	Paid gig without budget	Rejected
03	Past deadline at creation	Rejected
04	Keyword search	Correct matching gigs
05	Combined category/payment filters	Both applied
06	Apply to another user’s Open gig	Pending created
07	Apply to own gig	Rejected
08	Duplicate application	Rejected
09	Apply to expired gig	Rejected
10	Non-owner accepts	Rejected
11	Owner accepts one	Selected Accepted
12	Other Pending applicants	Automatically Rejected
13	Gig after acceptance	In Progress
14	Apply after acceptance	Rejected
15	Complete Open gig	Rejected
16	Owner completes In Progress	Completed
17	Outsider reviews	Rejected
18	Owner reviews applicant	Saved
19	Applicant reviews owner	Saved
20	Duplicate review	Rejected
21	Rating outside 1–5	Rejected
22	Profile rating	Calculated correctly
23	Refresh after mutation	Data persists
24	Run seed script	All required valid counts

13. Judge Demonstration

• Switch to Sara and post a Paid Requesting React gig.

• Switch to Ali, search/filter Programming gigs and apply.

• Switch to Sara; review applicants and accept Ali.

• Show all other Pending applications rejected, gig In Progress and new applications blocked.

• Sara completes gig; both users submit independent reviews.

• Show updated calculated reputation and completed-gig counts.

14. Future Enhancements

After MVP: verified university email, in-app messaging, payment gateway, AI matching, notifications, portfolio uploads, moderation/dispute handling and multi-university support. These are presentation scope only, not hackathon deliverables.

15. Final Deliverables

• Working responsive React application and Express mock API.

• Persistent, valid seed data with required minimum counts.

• All five milestones functioning through API-backed UI.

• Complete GitHub repository and README with install, seed, run and demo steps.

• Verified end-to-end demo; deployed link only if required by organizers.

16. Master AI Implementation Instruction

Build CampusConnect strictly against this PRD. Use React/Vite/Tailwind/React Router, Express and a persistent JSON mock API. Implement every P0 requirement and all five milestones. Keep consistent schemas and response formats, enforce rules on backend, supply complete executable code (not pseudocode), provide seed script and README, and avoid placeholder buttons or hardcoded frontend marketplace arrays. Definition of done: Student A posts → Student B applies → A accepts B → others rejected → In Progress → A completes → both review → profile ratings update. Prioritize reliable integration and working workflows over decorative additions.